

MACHINE LEARNING

(sanjay chauhan)

1. What is the difference between supervised, unsupervised?

The primary difference between **supervised** and **unsupervised learning** lies in the presence or absence of labeled data during training and the type of problems they aim to solve.

In supervised learning, we have input and output data. The model learns from labeled data — for example, predicting house prices based on area, location, etc.

In unsupervised learning, we only have input data and no labels. The model finds patterns or groups in the data — like customer segmentation based on shopping behavior.

2. Explain the bias-variance tradeoff.

Bias-variance tradeoff is about balancing two types of errors in a model.

Bias: the inability of a model to fit the training data.

High bias typically results from **underfitting**, where the model is too simple to capture the underlying patterns in the data.

Variance: Variance refers to the error introduced by the model's sensitivity to small fluctuations in the training data.

High variance typically results from **overfitting**, where the model captures noise or random fluctuations in the training data rather than the true underlying patterns.

Total Error = Bias² + Variance + Irreducible Error

Reducing bias often increases variance, and vice versa.

The goal is to find the right balance where the model generalizes well to unseen data.

3. What are overfitting and underfitting? How can you prevent them?

Overfitting happens when the model learns the training data too well, including noise. It does great on training data but poorly on test data.

Underfitting is when the model is too simple and can't learn the patterns, so it performs badly on both training and test data.

To prevent overfitting:

- Use simpler models
- Use regularization (like L1 or L2)
- Use more training data
- Use cross-validation

To prevent underfitting:

- Use a more complex model
- Train for more epochs
- Reduce regularization if it's too strong

4. How do you choose the right evaluation metric for a model?

Choosing the right metric depends on the **type of problem** you're solving and what matters most in your case:

- For **classification**:
 - Use **accuracy** if classes are balanced.
 - Use **precision, recall, or F1-score** if there's class imbalance.
 - Use **ROC-AUC** to check how well the model separates classes.
- For **regression**:
 - Use **MAE** if you care about average absolute error.
 - Use **MSE** or **RMSE** if large errors are a bigger concern.
 - Use **R² score** to see how well your model explains the data.

5. What is the curse of dimensionality? How can you address it?

The **curse of dimensionality** means that as the number of features (dimensions) increases, the data becomes sparse, and models may perform poorly because they can't learn patterns well.

How to handle it:

- Use **feature selection** to keep only important features.
- Use **dimensionality reduction** like PCA.
- Remove irrelevant or highly correlated features.
- In short, reduce the number of features to help the model focus better.

6. What are some common ways to handle missing data?

Here are some common ways to handle missing data:

Remove rows with missing values (if few and not important).
Fill with mean, median, or mode (works for numerical or categorical).
Use forward/backward fill for time series data.
Use models (like KNN or regression) to predict missing values.
Use a special category for missing in categorical data (like “Unknown”).

7. Explain the difference between parametric and non-parametric models.

Parametric models assume a fixed form or equation (like linear regression). They learn a few parameters and are faster but may not fit complex data well.

Non-parametric models don't assume any fixed form. They can adapt to the data shape (like decision trees or KNN) and handle complex patterns, but they need more data and are slower.

In short:

- **Parametric = simple, fast, needs assumptions**
Non-parametric = flexible, slower, no strong assumptions

8. What are hyperparameters, and how do you tune them?

Hyperparameters are settings you choose before training a model — like learning rate, number of trees in Random Forest, or k in KNN. They are not learned from the data.

To tune them, you can use:

Grid Search: try all combinations

Random Search: try random combinations

Use **cross-validation** to test each combination and pick the one with the best performance.

9. How do you handle imbalanced datasets?

To handle imbalanced datasets, you can:

1. **Resample the data**:
Oversample the minority class (like using SMOTE)
Undersample the majority class
2. **Use proper metrics** like precision, recall, F1-score, or ROC-AUC instead of accuracy.

3. **Use class weights** in your model to give more importance to the minority class.

10. What is the difference between bagging and boosting?

Both **Bagging** and **Boosting** are ensemble learning techniques used to improve the performance of machine learning models by combining multiple base models (often decision trees). However, they differ in how they build the ensemble and how they combine the models.

Bagging (like Random Forest):

Trains **multiple models in parallel** on different random subsets.
Aims to **reduce variance**.
Each model gets equal vote in the final result.

Boosting (like XGBoost, AdaBoost):

- Trains models **one after another**, where each model fixes the errors of the previous one.
- Aims to **reduce bias**.
- Final model gives more weight to better learners.

11. What is the kernel trick in Support Vector Machines (SVM)?

The **kernel trick** lets SVM handle non-linear data by mapping it into a higher-dimensional space **without actually doing the transformation**.

Instead of transforming features manually, the kernel trick uses a **function** (like polynomial or RBF) to calculate the similarity between data points in that high-dimensional space.

This way, SVM can draw a straight line (hyperplane) in the new space, which becomes a **non-linear boundary** in the original space.

12. What is gradient descent, and how does it work?

Gradient Descent is an optimization algorithm used to minimize the cost (or loss) function in machine learning models. It is commonly used in training models such as linear regression, logistic regression, neural networks, and other machine learning algorithms. The goal of gradient descent is to find the optimal parameters (weights) that minimize the error in the model's predictions.

13. Explain Principal Component Analysis (PCA).

Principal Component Analysis (PCA) is a statistical technique used for **dimensionality reduction** while preserving as much variance (information) in the

data as possible. PCA transforms a set of correlated features into a smaller set of uncorrelated features called **principal components**. These principal components are ordered such that the first few components retain most of the variation present in the original data.

14. What is cross-validation, and why is it important?

Cross-validation is a technique to test how well a model will perform on unseen data.

The most common type is **k-fold cross-validation**:

You split the data into k parts (folds).
Train on $k-1$ parts and test on the remaining one.
Repeat k times and take the average score.

It's important because it gives a **more reliable estimate** of model performance and helps prevent **overfitting** or **underfitting**.

15. What are some common techniques for feature selection?

1. Filter Methods:

Correlation Coefficient, Chi-Square Test, ANOVA (Analysis of Variance), Variance Thresholding

2. Wrapper Methods:

Forward Selection, Backward Elimination, Recursive Feature Elimination (RFE)

3. Embedded Methods:

L1 Regularization (Lasso Regression), Decision Trees and Random Forests (Features importance)

4. Dimensionality Reduction Methods:

Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

16. What is the difference between feature selection and feature extraction?

Feature selection means choosing the most important features from the existing ones — you **keep** useful features and drop the rest.

Feature extraction means creating **new features** by combining or transforming the original ones — like using **PCA** to reduce dimensions.

In short:

Feature selection = pick best original features

Feature extraction = make new features from existing ones